

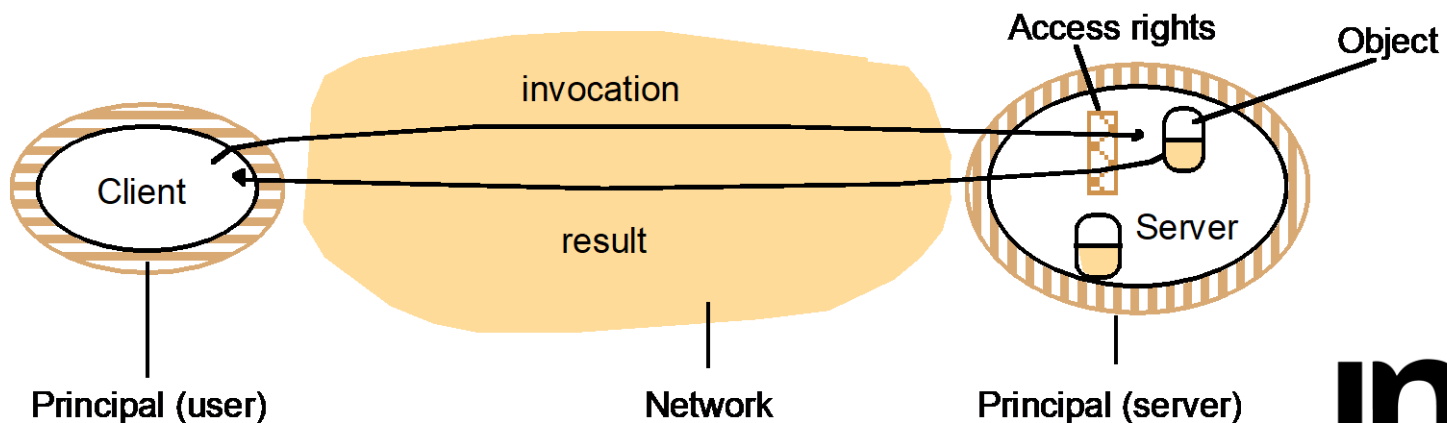
INF01151 – Sistemas Operacionais II

Aula 27 - Segurança

Prof. Alberto Egon Schaeffer Filho

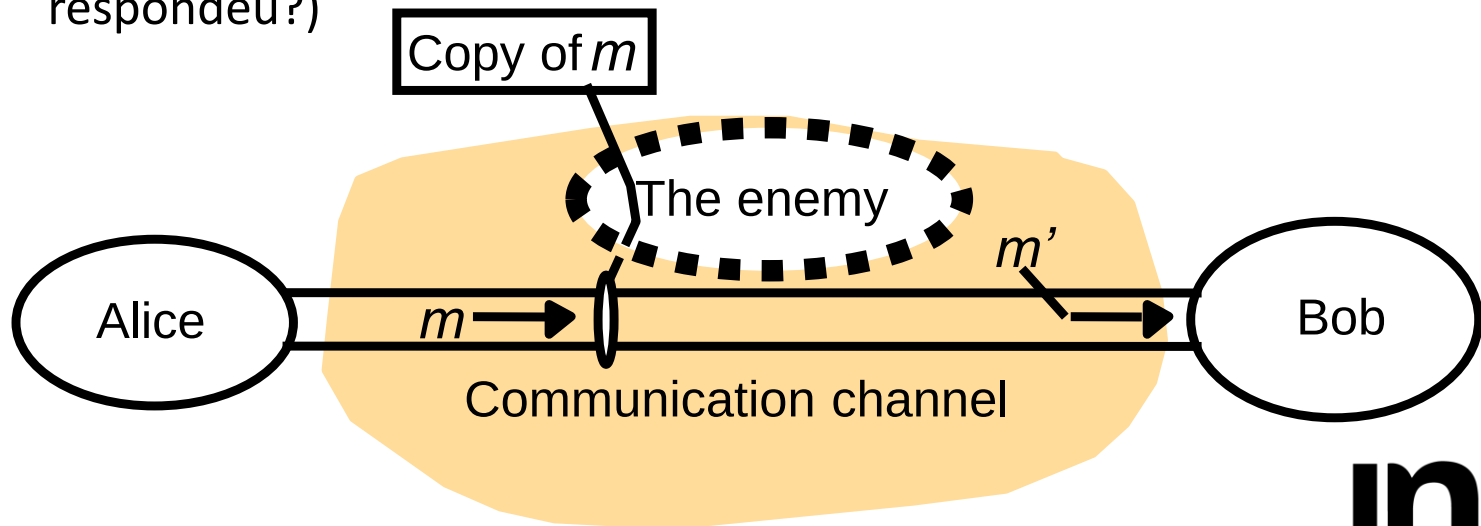
Introdução

- Sistemas distribuídos são caracterizados por compartilhar recursos
 - Necessário manter privacidade, integridade e disponibilidade
- Objetivo:
 - Tornar seguro os **recursos** e os **canais de comunicação** usados em sua interação, protegendo-os contra acessos não autorizados
 - Controle de acesso, cifragem do canal de comunicação, identificação de quem solicita e de quem responde (*principals*)



Ameaças e ataques

- Processos se comunicam através de mensagens
 - É possível que usuário malicioso copie, altere e injete mensagens na rede que são transmitidas entre *Alice* e *Bob*
- Um processo precisa saber com quem ele está se comunicando
 - Servidor: é feito para atender requisições de clientes (quem é o cliente?)
 - Cliente: é feito para receber uma mensagem de resposta (quem respondeu?)



Principais ameaças

- Vazamento (*leakage*)
 - Aquisição de informações por usuários não autorizados
- Falsificação (*tampering*)
 - Modificação não autorizada da informação
- Vandalismo (*vandalism*)
 - Interferência na operação correta de um sistema, sem ganho para o invasor

Principais ataques

- Intromissão (*eavesdropping*)
 - Obter cópias de mensagens sem autorização
- Mascaramento (*masquerading*)
 - Enviar/receber mensagens com a identidade de outro participante, sem sua autorização
- Falsificação de mensagem (*message tampering*)
 - Interceptar mensagens e alterar seu conteúdo, antes de passá-las para o destinatário desejado (e.g.: man-in-the-middle)
- Reprodução (*replaying*)
 - Armazenar mensagens interceptadas e enviá-las em uma data posterior.
 - Eficiente mesmo em certos casos onde a mensagem é autenticada e cifrada
- Negação de serviço (*denial-of-service*)
 - Saturar o canal ou recursos com mensagens para impedir (negar) o acesso de outras pessoas

Aspectos de segurança em sistemas distribuídos

- Em segurança da informação é importante garantir três propriedades:
 - **Confidencialidade:** não disponibilizar (conteúdo das) mensagens p/ processos não autorizados
 - **Integridade:** prevenir que dados sejam modificados indevidamente
 - **Disponibilidade:** garantir acesso ao serviço sempre que esse for necessário
- Porém há outras duas correlacionadas:
 - **Autenticidade:** identificar as partes envolvidas (possivelmente ambos)
 - **Não repúdio:** provar que a mensagem (ação) foi feita por seu remetente
- Uso de criptografia auxilia na obtenção dessas propriedades
 - Base para autenticação, não-repúdio, confidencialidade e integridade

Criptografia

- Criptografia é a base para os mais populares mecanismos de segurança
 - Uso primário: processo de codificação de uma mensagem de forma a ocultar seu conteúdo
 - Uso de algoritmos para cifrar e decifrar uma mensagem
 - Baseado no conhecimento de segredos (chaves)
 - Também pode ser usado para autenticação e assinaturas digitais
- Duas classes de algoritmos de criptografia
 - Criptografia de chave secreta (simétrica)
 - Emissor e destinatário compartilham um segredo (chave secreta)
 - Criptografia de chave pública (assimétrica)
 - Baseado em um par de chaves para cada usuário: uma pública, outra privada
 - A chave privada é de conhecimento apenas de seu possuidor
 - A chave pública é de conhecimento de todos
 - Requer de 100-1000 vezes mais poder de processamento para decifrar mensagem comparado com algoritmos de chave secreta

Criptografia: princípio básico de funcionamento

- Codificação (cifrar):
 - Determinar uma função f que a partir de um texto puro (P) se obtém um texto cifrado (C) com ajuda de uma chave K_e

$$f(K_e, P): P \mapsto C$$

- Decodificação (decifrar):
 - Permitir a operação inversa a partir de uma chave K_d

$$f(K_d, C): P \quad \text{ou} \quad f(K_d, f(K_e, P)) = P$$

$$K_e = K_d \rightarrow \text{simétrica}$$

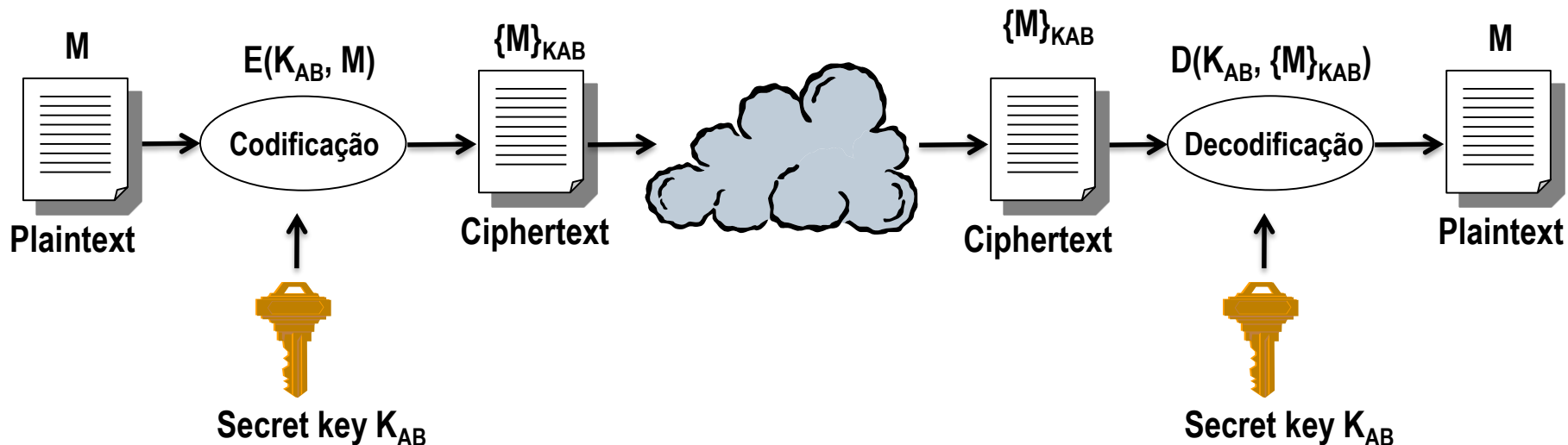
$$K_e \neq K_d \rightarrow \text{assimétrica}$$

Criptografia: principais usos

- Criptografia desempenha **três papéis principais** na implementação de segurança:
 - **Confidencialidade e integridade** (*secrecy and integrity*): proteger informação quando ela for transmitida através da rede
 - Por exemplo, durante transmissão na rede que pode ser vulnerável à *eavesdropping* e *tampering* (papel original militar)
 - Explora a propriedade de que algo cifrado com uma chave só pode ser decifrado por alguém que possuir a chave correspondente
 - **Autenticação** (*authentication*): receptor que consegue decifrar mensagem com uma dada chave, pode verificar se ela está correta (e.g., *checksum*)
 - Seguro inferir que o emissor tinha a chave de cifragem, assim deduzindo identidade do emissor
 - Se chaves foram mantidas em segredo, sucesso na decifragem autentica o emissor
 - **Assinaturas digitais** (*digital signatures*): emula papel de assinaturas convencionais
 - Garante que mensagem é uma cópia inalterada daquela produzida na origem

Cenário: confidencialidade com chave secreta

- Alice deseja enviar uma informação de forma confidencial para Bob, utilizando uma chave compartilhada K_{AB}



Assumindo que a chave não foi comprometida, e que o algoritmo utilizado seja forte o bastante. Problemas?

Problema 1: Como Alice enviou chave secreta K_{AB} para Bob?

Problema 2: Como Bob sabe que $\{M\}_{KAB}$ não é uma cópia de uma mensagem enviada por Alice anteriormente?
(e.g., requisição para pagar alguém)

Confidencialidade

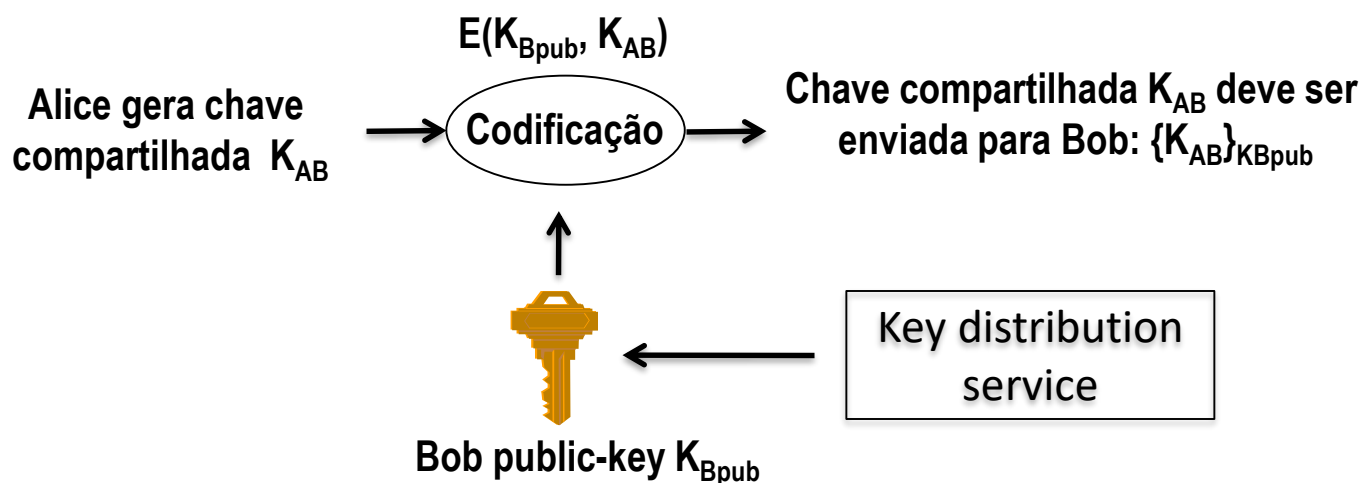
- Confidencialidade é resolvida cifrando a mensagem
- Normalmente:
 - Criptografia assimétrica é usada para negociar uma chave de sessão (chave simétrica) que é usada para cifrar os dados

	Criptografia simétrica	Criptografia assimétrica
Custo computacional	Baixo	Alto
Qtde de chaves	$n*(n-1)/2$	2 por participante ($2*n$)
Distribuição/armazenamento	Difícil	Fácil (certificado digital)
Uso típico	Cifragem de grande quantidade de dados	Autenticação, não repúdio

Protocolos híbridos de criptografia

Cenário: autenticação com chave pública

- Bob gerou um par de chaves pública/privada, e Alice e Bob desejam estabelecer uma chave compartilhada K_{AB}



Somente Bob pode recuperar K_{AB} , utilizando sua chave privada para decifrar mensagem

Problemas?

Problema: vulnerável a ataque man-in-the-middle. Attacker pode interceptar a requisição inicial de Alice, e enviar para Alice a chave pública do Attacker. Dessa forma, todas as mensagens que Alice pensa trocar com Bob são interceptadas pelo usuário malicioso

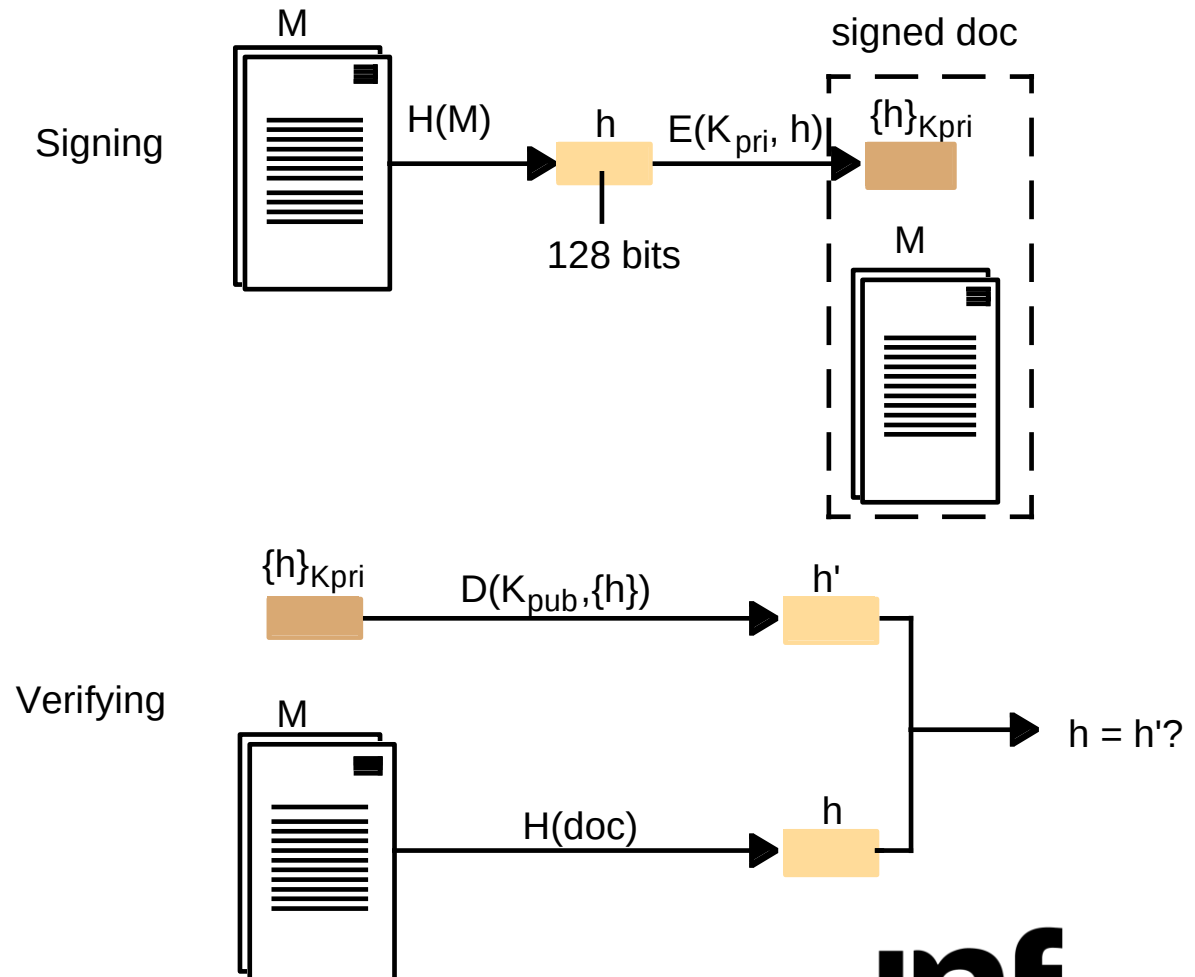
Solução: garantir que a chave pública de Bob (certificado) foi assinado por alguma autoridade bem conhecida

Assinatura digital

- Objetivo é garantir a autenticidade de uma mensagem (documento)
 - Convencer receptor que emissor assinou o documento + integridade
 - Associar identidade da origem à sequência de bits que constitui documento
- Técnicas para assinatura digital baseadas em criptografia:
 - **Digital signing with public-keys**
 - Simples, não requer comunicação entre receptor e origem para validação
 - Participante **A** assina documento **M** utilizando sua chave privada K_{pri}
 - Geralmente utilizam Digest functions (secure hash functions)
 - Denotadas por $H(M)$, e projetadas para que $H(M) \neq H(M')$ se $M \neq M'$
 - **Digital signatures with secret keys**
 - Solução menos conveniente que método utilizando chave pública/privada
 - Tecnicamente também pode ser utilizada para gerar a assinatura, mas o desafio é compartilhar essa chave
 - Vantagem: desempenho (não faz encryption)

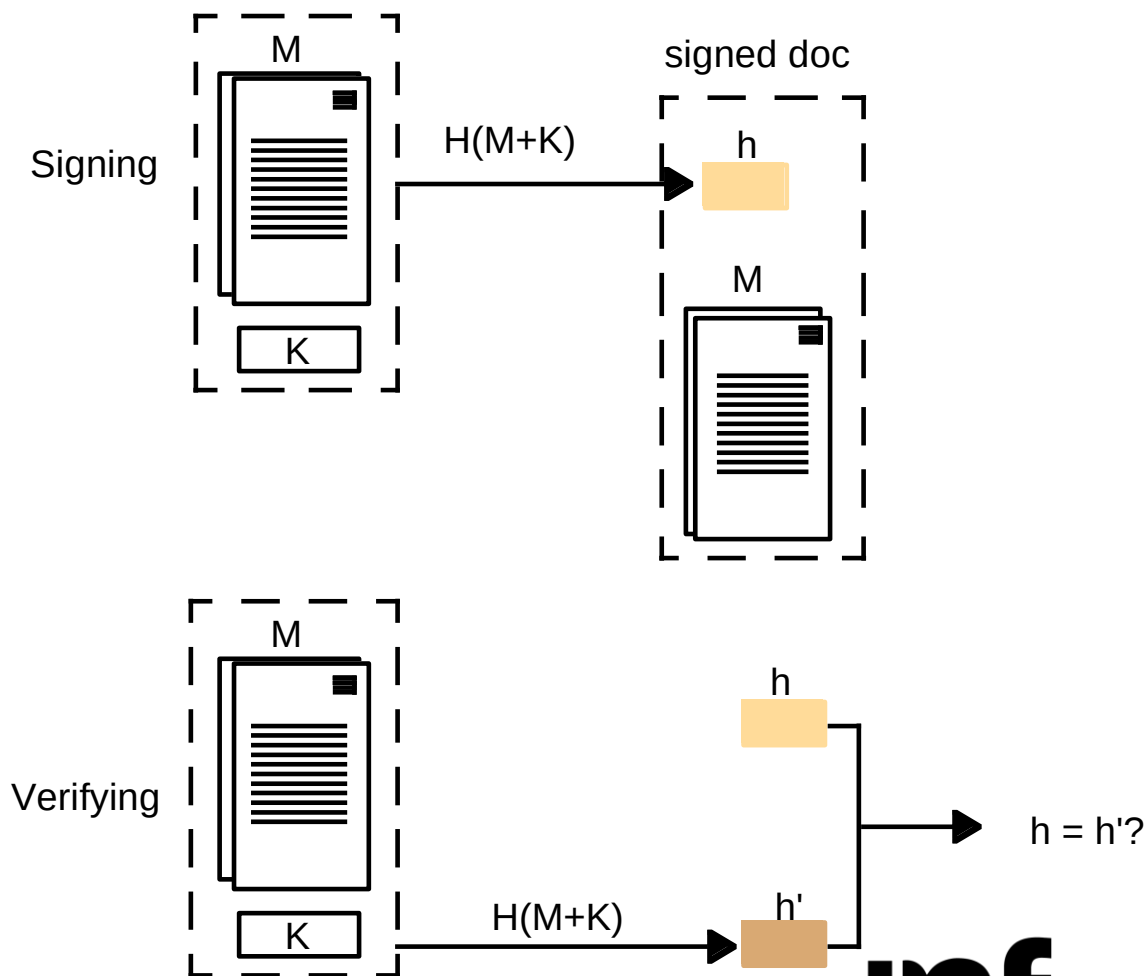
Assinatura digital usando chave pública

1. **A** gera um par de chaves K_{pub} e K_{pri} , e publica chave pública K_{pub}
2. **A** calcula digest de **M**, $H(M)$ usando uma função **H** conhecida por ambos
3. **A** cifra $H(M)$ utilizando chave privada K_{pri} , produzindo assinatura $S = \{H(M)\}_{K_{pri}}$
4. **A** envia mensagem assinada **M, S** para **B**
5. **B** decifra **S** usando K_{pub} e computa o hash de **M**, $H(M)$. Se forem iguais, assinatura é válida



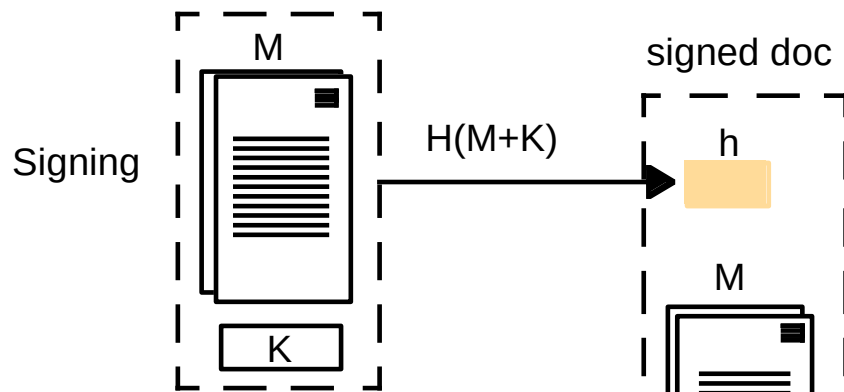
Assinatura digital usando chave secreta

1. **A** gera uma chave **K** para assinar documentos, e distribui ela “*de alguma forma segura*” para outros participantes
2. **A** concatena **M+K**, e utiliza função de hash para calcular digest **$h = H(M+K)$**
3. **A** envia o documento assinado **M, h** (chave **K** não é comprometida pela divulgação de **h**)
4. **B** concatena a chave secreta **K** com o documento recebido **M**, e computa **$h' = H(M+K)$** . Se forem iguais, assinatura é válida



Assinatura digital usando chave secreta

1. **A** gera uma chave **K** para assinar documentos, e distribui ela “*de alguma forma segura*” para outros participantes
2. **A** concatena **M+K**, e utiliza função de hash para calcular

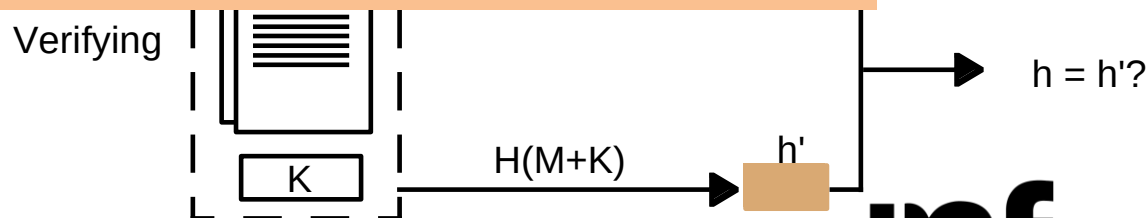


Propriedades de uma secure digest function $h = H(M)$:

1. Dado **M**, é fácil computar **h**
2. Dado **h**, é difícil computar **M**
3. Dado **M**, é difícil encontrar outra mensagem **M'**, tal que $H(M) = H(M')$

Exemplos: **MD5, MD6, SHA-2, SHA-3**

M, e computa $h' = H(M+K)$. Se forem iguais, assinatura é válida



Protocolos de autenticação

- Autenticação user + password é inconveniente em sistemas distribuídos
 - Necessário uma base de dados com todos possíveis usuários
 - Autenticação a cada ação do usuário no sistema
 - Solução é definir protocolos de autenticação ...
- Princípio básico
 - Identificar se o usuário é quem ele diz ser e isso é feito com base em algo que só ele conhece ou possui: chave
 - Protocolos de autenticação
 - **Needham-Schroeder**
 - **Kerberos**

Protocolo de autenticação Needham-Schroeder

- Solução para autenticação e distribuição de chaves
 - Objetivo: permitir comunicação segura entre processo A e um processo B
- Baseado em um servidor de autenticação
 - Servidor mantém uma tabela com o nome e uma **chave secreta** para cada participante
 - Usada para autenticar o cliente p/ servidor de autenticação
 - Usada para transmissão segura entre o cliente e servidor de autenticação
 - **Problema**: como distribuir a **chave compartilhada**?
 - Servidor gera **chaves compartilhadas** para para pares de clientes A e B
 - Protocolo baseado na geração e transmissão de **tickets** pelo servidor de autenticação
 - Ticket: mensagem cifrada que contém chave secreta para comunicação entre A e B
- Existe também versão baseada em chave pública...

Protocolo Needham-Schroeder: mensagens

Header	Message	Notes
1. A→S:	A, B, N_A	A requests S to supply a key for communication with B.
2. S→A:	$\{N_A, B, K_{AB}, \{K_{AB}, A\}_{K_B}\}_{K_A}$	S returns a message encrypted in A's secret key, containing a newly generated key K_{AB} and a 'ticket' encrypted in B's secret key. The nonce N_A demonstrates that the message was sent in response to the preceding one. A believes that S sent the message because only S knows A's secret key.
3. A→B:	$\{K_{AB}, A\}_{K_B}$	A sends the 'ticket' to B.
4. B→A:	$\{N_B\}_{K_{AB}}$	B decrypts the ticket and uses the new key K_{AB} to encrypt another nonce N_B .
5. A→B:	$\{N_B - 1\}_{K_{AB}}$	A demonstrates to B that it was the sender of the previous message by returning an agreed transformation of N_B .

Kerberos

- Baseado em dois serviços

- Kerberos Authentication Server (KAS)

- Autentica o usuário e fornece um ticket para acessar o TGS

- Ticket Granting Server (TGS)

- Fornece tickets para usuários empregarem outros serviços

Ticket p/ cliente C acessar servidor S:

$\{C, S, t_1, t_2, K_{CS}\}_{K_S}$

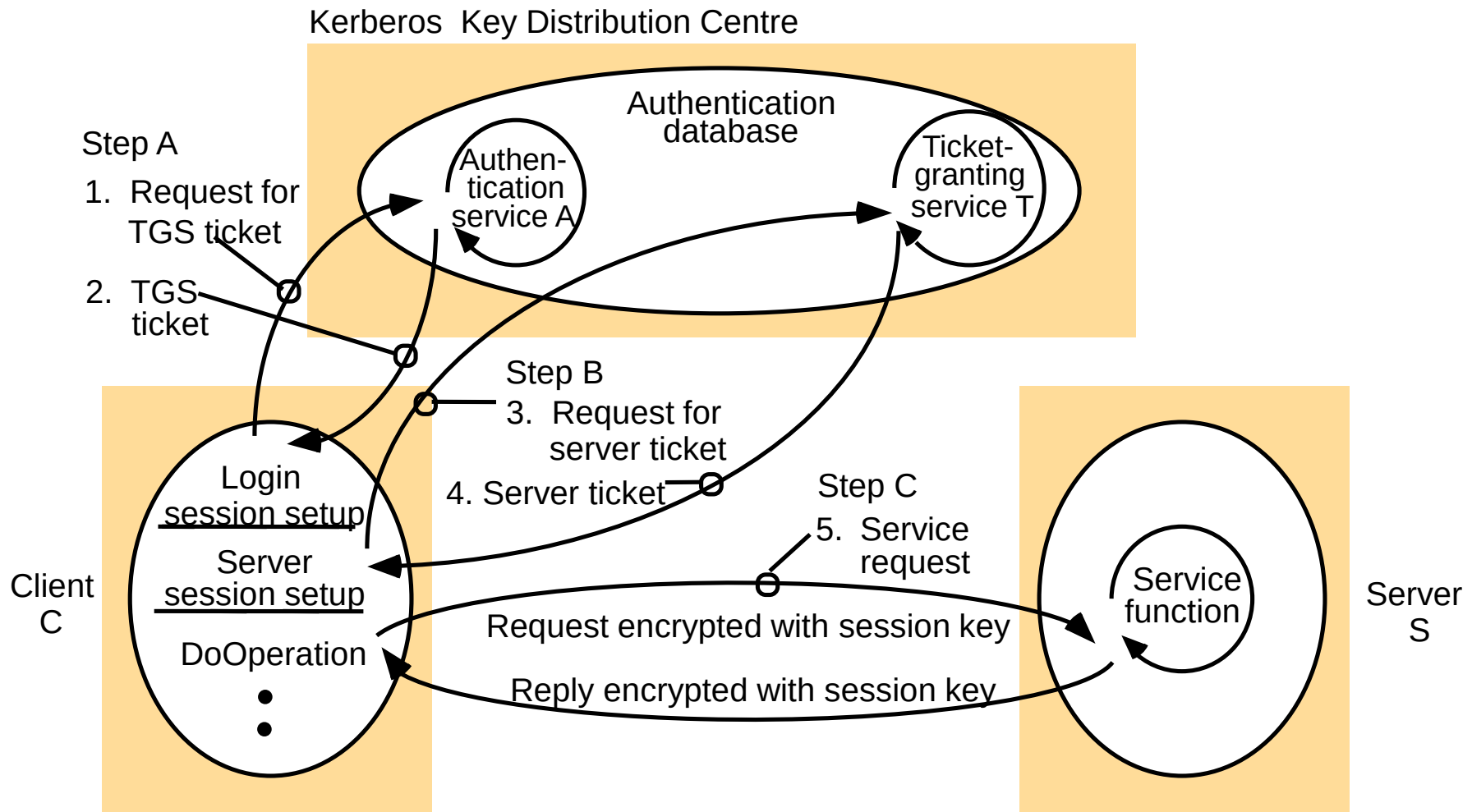
Authenticator de cliente C
p/ acessar servidor S:

$\{C, t\}_{K_{CS}}$

- Arquitetura Kerberos define 3 elementos de segurança:

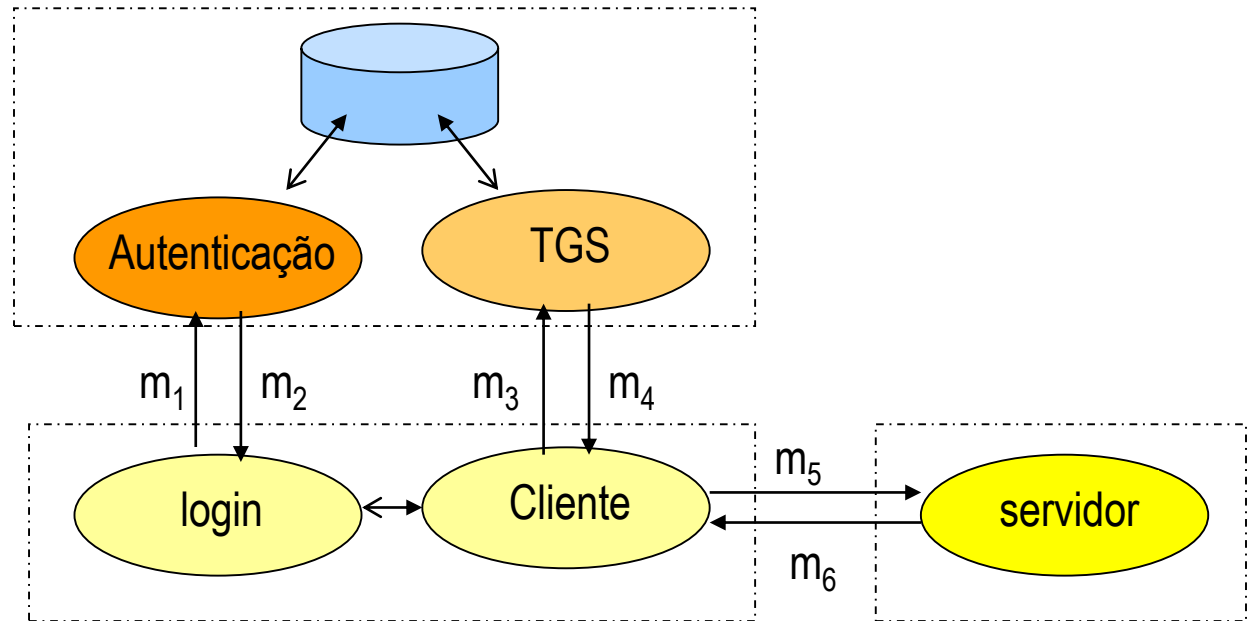
- **Ticket**: Implementa noção de **capability** (privilégio p/ executar ação). Token emitido pelo TGS, que deve ser apresentado para utilização de um serviço S
 - Indica que processo foi recentemente autenticado pelo Kerberos
 - Inclui *expiry time* e *session key* gerada
 - **Authenticator**: token construído pelo cliente para provar sua identidade. Só pode ser utilizado uma vez para acessar serviço S
 - Contém nome do cliente, timestamp e é encriptado com a session key
 - **Session key**: gerada randomicamente pelo Kerberos e emitida para que cliente se comunique com um determinado serviço. Usada para cifrar o authenticator
 - Também usada para cifrar a comunicação com o serviço S (se necessário)

Arquitetura do sistema Kerberos



Protocolo de autenticação do Kerberos

ID_g : id. TG
 ID_c : id. cliente
 ID_s : id. servidor
 N_i : nonce
 K_c : chave secreta cliente
 K_s : chave secreta servidor
 K_g : chave secreta TGS
 K_1 : chave sessão TGS
 K_2 : chave sessão servidor
 T_{si} : tempo inicial ticket
 T_{ei} : tempo final ticket
 T_i : timestamp



$m_1 = (ID_c, N_1)$
 $m_2 = C_2 = E((N_1, K_1, C_1), K_c)$
 $m_3 = (ID_s, N_2, C_1, C_3)$
 $m_4 = C_5 = E((N_2, K_2, C_4), K_1)$
 $m_5 = (C_4, C_6)$
 $m_6 = C_7 = E(T_3, K_2)$

$C_1 = E((ID_c, ID_g, T_{s1}, T_{e1}, K_1), K_g)$
 $C_3 = E((ID_c, T_1), K_1)$
 $C_4 = E((ID_c, ID_s, T_{s2}, T_{e2}, K_2), K_s)$
 $C_6 = E((ID_c, T_2), K_2)$
 $T_3 = T_2 + 1$

Leituras adicionais

- Coulouris, G.; Dollimore, J.; Kindberg, T. and Blair, G.—
“*Distributed Systems: Concepts and Design*” (5th edition),
Addison Wesley, 2012
 - Capítulo 11

Secure Socket Layer (SSL)

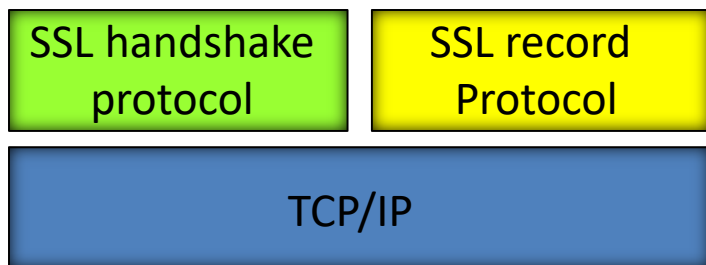
- Desenvolvido originalmente pela Netscape Corporation
 - Versão estendida de SSL foi adotada como padrão na internet
 - Transport Layer Security (TLS), descrito na RDC 2246
 - TLS suportado pela maioria dos browsers, utilizado em e-commerce
- Projetado para fornecer cifragem de dados entre um cliente e um servidor web
 - Não é limitado a web, pois oferece uma API de programação para aplicação



Exemplos de aplicações na Internet: HTTPS (443); SSMTP(465); POP (995); IMAP (993); etc

Arquitetura SSL

- Organizado em dois protocolos básicos:
 - Protocolos específicos empregado no gerenciamento do SSL
 - Usado no estabelecimento
 - SSL Record Protocol
 - Empregado na utilização de uma conexão segura
- Possui ainda um protocolo para troca de chaves e alerta



- Conjunto de algoritmos de cifragem e opções distintas
- Compactação
- Restrições quanto a chaves e uso

Características SSL

- Apresentação mútua
 - Negociação do algoritmo de cifragem (DES, IDEA, etc) e de chave de sessão
 - Autenticação do servidor ao cliente e do cliente ao servidor (opcional)
- Autenticação do servidor
 - Permite confirmação da identidade do servidor
 - Cliente mantém listas de ACs com suas chaves públicas
 - Obtém certificado do servidor (isto é, sua chave pública)
 - Verifica autenticidade do certificado (assinatura de um AC)
- Autenticação do cliente (opcional)
 - Análoga ao servidor cliente envia certificado ao servidor
- Sessão SSL
 - Transferência de dados (cifrado)

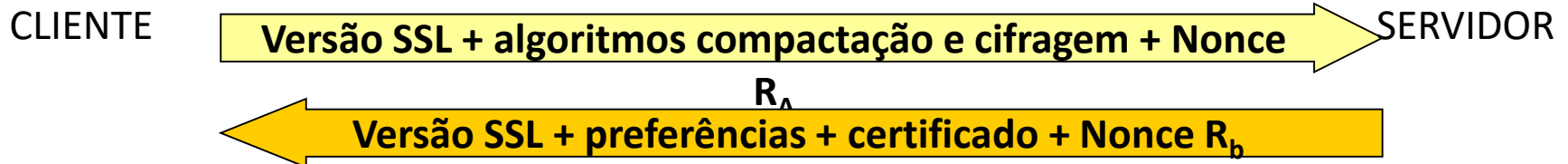
Características do SSL

- Prove autenticação e confidencialidade
- Emprega cifragem assimétrica (chave pública/privada) e chaves de sessão
- Duas partes:
 - SSL handshake protocol
 - Fornece a autenticação dos pares (criptografia assimétrica)
 - Usado para gerar uma chave de sessão (chave privada)
 - Uso de certificados digitais
 - SSL record protocol
 - Fornece a confidencialidade
 - Usado para cifrar as mensagens
 - Autenticidade (integridade + autenticação) é via assinatura digital da mensagem

Protocolo *Handshake*

- Parte mais complexa do SSL.
- Possibilita a autenticação mútua de clientes e servidores.
- Negocia codificação, algoritmo MAC (*Message Authentication Code*) e chaves de criptografia.
- Empregado antes que qualquer dado seja transmitido.

Funcionamento SSL no https (*handshake*)



1. AC do certificado está na lista?

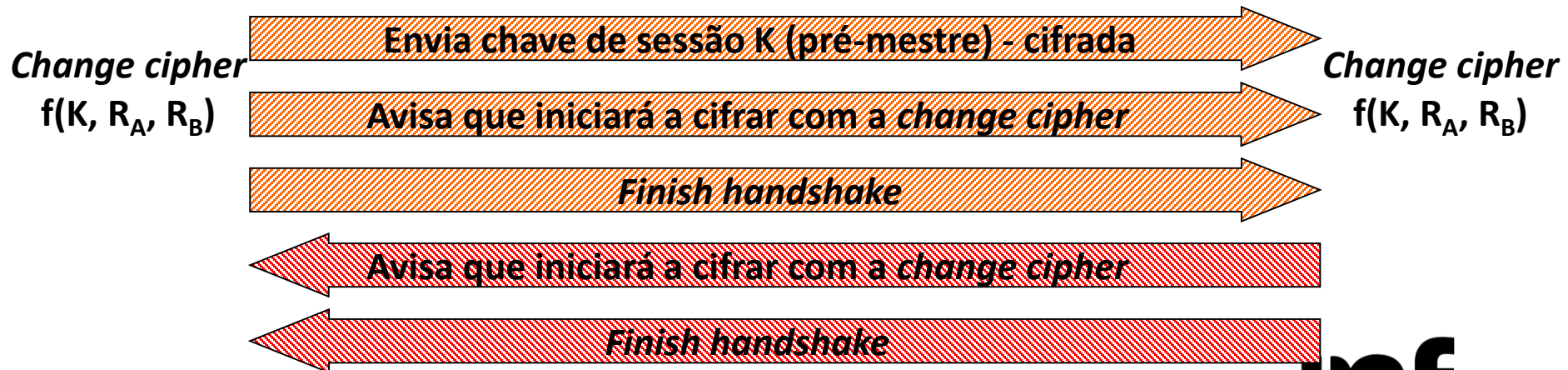
NÃO: avisa usuário

SIM:

- valida certificado com a chave pública da AC e obtém chave pública do servidor

Opcional: solicitação do certificado do cliente

2. Gera chave de sessão K



Funcionamento do SSL (envio de dados)

